PROJECT:

RATED
ZK

FOR: ZK-LORA PRIVACY LAYER

ZERO-KNOWLEDGE AI-TO-AI MESH NETWORKS

UNDER DECENTRALIZED incentivization PROTOCOL

Proposal Type:	Zcash Community Grants — Research & Development
AI POD:	zymatica.space, astronautshe.com, Devs One + 9 other AI dev agents
HUMANS:	LEAD ARCHITECT: Danny Bouldiez + 2 human Devs
Roles:	zymatica (Lead Cryptographer), astronautshe (Edge Systems Engineer), Devs One (AI Swarm)
Platform:	Zcash Shielded Pool, Raspberry Pi OS, Semtech SX1302/1303 HAL
Status:	Milestone 1 Verified & Achieved // Milestone 2 Pending Approval // Milestone 3 Pending Approval

ZK-LORA: ZERO-KNOWLEDGE PROOFS FOR PRIVATE AI-TO-AI MESH NETWORKS

A Bitcoin-Style Identity System with Zcash Shielded Privacy for LoRaWAN Communications

■ Executive Summary

Traditional LoRaWAN communication has a critical privacy gap: lack of end-to-end user-layer encryption, open device tracking, and vulnerability to behavioral mapping. **ZK-LoRa (Zero-Knowledge LoRa)** introduces a secure, decentralized privacy layer for AI-to-AI mesh networks. By combining Bitcoin's public-key cryptography (secp256k1) with Zcash's zero-knowledge proofs (Groth16 on BN128) and shielded transactions, ZK-LoRa allows autonomous edge nodes to communicate over RF without revealing their hardware identities.

This project represents a massive opportunity for the Zcash community to bridge digital privacy with physical hardware by leveraging existing DePIN infrastructure. The Helium network built a global RF infrastructure with over 980,000 registered on-chain hotspots. As Helium's reward structures and optimization proposals (such as **HIP-149**) evolve, a significant portion of these gateways have become underutilized, offline, or economically dormant.

ZK-LoRa provides a highly realistic, secondary utility for these pre-certified devices—including over 300,000 RAKwireless-manufactured hotspots (RAK V2, MNTD) equipped with Semtech SX1302/SX1303 concentrator chips and Raspberry Pi units. By running open-source packet forwarders alongside or in place of original firmware, operators can participate in private edge routing, verify zero-knowledge proofs on-chip in milliseconds, and earn shielded Zcash (ZEC) micropayments. Thanks to Zcash's multi-output transaction architecture, this payment split is completely programmable, allowing a custom percentage (e.g., 5% or 10%) to support the Zcash Foundation, with a 2% split supporting the developer treasury.

GitHub: Main Repository
zk-lora-privacy-layer

Covers: Core SX1302/3 HAL drivers, ECIES encryption, secp256k1 identity derivation, and full multi-language ports (Rust, Go, TS).
Achieves: Complete off-grid identity masking and end-to-end message privacy over public RF bands.

GitHub: Milestone 1 Workspace
Multi-Language Proofs

Covers: secp256k1 key generation, Ripemd160/SHA256 address derivation, Groth16 ZK-SNARK compiler, and proof generation/verification.
Achieves: Cryptographic proof of node legitimacy without revealing hardware or network identities.

GitHub: Milestone 2 Workspace
Zcash Mempool Scanner

Covers: Rust/Go/TS mempool scanners, shielded memo decryption via Incoming Viewing Keys (IVKs), and 2% developer fee validation.
Achieves: Zero-latency, off-chain routing authorization triggered instantly by pending Zcash mempool transactions.

GitHub: Milestone 3 Workspace
Multi-Hop Mesh & HAL

Covers: Multi-hop mesh routing, P2P data marketplace, and the 4 advanced security systems (ZK-PoD, HMAC filter, ToF bounding, and Neighbor Auditing).
Achieves: Absolute network resilience against Gorgon, Sybil, Eclipse, and Free Rider attacks.

■ The Challenge & The Solution

Deploying AI agents at the edge (on low-power hardware like Helium RAK miners or Raspberry Pi nodes) requires a robust, secure, and private communications channel. Traditional RF protocols fail in adversarial environments. Below is the comparative analysis of the corporate/traditional problems versus the ZK-LoRa solutions:

The Traditional Problem	The ZK-LoRa Solution
Identity Exposure: Every packet contains a static hardware ID (MAC/DevAddr) allowing eavesdroppers to track and map node locations.	**Bitcoin-Style Masking: Keypairs are generated locally. Nodes derive temporary 'LoRa phone numbers' which change dynamically via ZK proofs.**
Eavesdropping: Payloads are broadcasted in the clear or encrypted with static keys, vulnerable to decryption if keys are compromised.	**ECIES Encryption: Assures recipient-only decryption. Messages are encrypted with the recipient's public key, providing forward secrecy.**
Spam & DDoS: Low cost of RF transmission allows malicious actors to flood the channel, jamming legitimate agent communications.	**Proof-of-Useful-Work: Nodes must attach a valid ZK-SNARK proof. The computation acts as a spam-prevention puzzle.**
Uncompensated Relaying: Gateways must route packets for free, leading to central dependency or lack of coverage.	**Zcash Shielded Rewards: Gateways are compensated in ZEC. The transaction memo decrypts to match the packet reference.**

■ System Architecture: Identity & Encryption

Layer 1: Elliptic Curve Identity Derivation

ZK-LoRa implements a decentralized identity system inspired by Bitcoin. Each agent generates an ECDSA keypair using the *secp256k1* elliptic curve. The public key is hashed using SHA-256 followed by RIPEMD-160 (HASH160) to derive a short, unique 8-character hex identifier, formatted as a 'LoRa phone number':

terminal

```
Private Key (256-bit secret)
   ↓ (secp256k1 elliptic curve multiplication)
Public Key (65-byte uncompressed)
   ↓ (HASH160: SHA-256 + RIPEMD-160)
LoRa Phone Number: AGENT-7F3A9B2C@zymatica.space
```

Layer 2: Recipient-Only ECIES Encryption

To ensure absolute confidentiality over public RF bands, payloads are encrypted using the Elliptic Curve Integrated Encryption Scheme (ECIES). The sender uses the recipient's public key to derive a shared secret, encrypts the payload using AES-128-GCM, and attaches the ephemeral public key to the frame. Only the holder of the recipient's private key can decrypt the message.

terminal

```
// Local Identity Keyfile Format (~/.zyMatica/keys/researcher-1.json)
{
  "agent_name": "researcher-1",
  "phone_number": "71E457CE",
  "private_key": "6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b",
  "public_key": "04a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2...",
  "zyMatica_address": "AGENT-71E457CE@zymatica.space"
}
```

■ Layer 3: Zero-Knowledge Proofs

The core privacy mechanism of ZK-LoRa is the decoupling of authentication from identity. Instead of broadcasting their public key or phone number (which would allow tracking), the agent generates a Groth16 ZK-SNARK proof. This proof mathematically demonstrates that the sender knows a valid private key corresponding to a registered identity in the network's authorized registry, without revealing the private key or the public key itself.

AgentValidityProof.circom

```
// ZK-SNARK Agent Validity Circuit (AgentValidityProof.circom)
pragma circom 2.0.0;
include "node_modules/circomlib/circuits/poseidon.circom";
include "node_modules/circomlib/circuits/babyjubjub.circom";
template AgentValidityProof() {
    signal input private_key;      // Witness (Secret Private Key)
    signal input public_key_hash;  // Public Input (Registered Identity Hash)
    signal output valid;           // 1 if valid, 0 if invalid
    // Derive public key on BabyJubjub curve
    component derive_pubkey = BabyJubjubDerive();
    derive_pubkey.private_key <== private_key;
    // Hash the derived public key using Poseidon
    component hasher = Poseidon(2);
    hasher.inputs[0] <== derive_pubkey.x;
    hasher.inputs[1] <== derive_pubkey.y;
    // Enforce that the hash matches the public input
    hasher.out === public_key_hash;
    valid <== 1;
}
```

■ Shielded Micropayment Incentives

The Zcash Shielded Micropayment mechanism is the economic engine of ZK-LoRa. It solves the biggest problem in decentralized radio networks: *How do you pay gateways to route your data without revealing who you are or where you are located?*

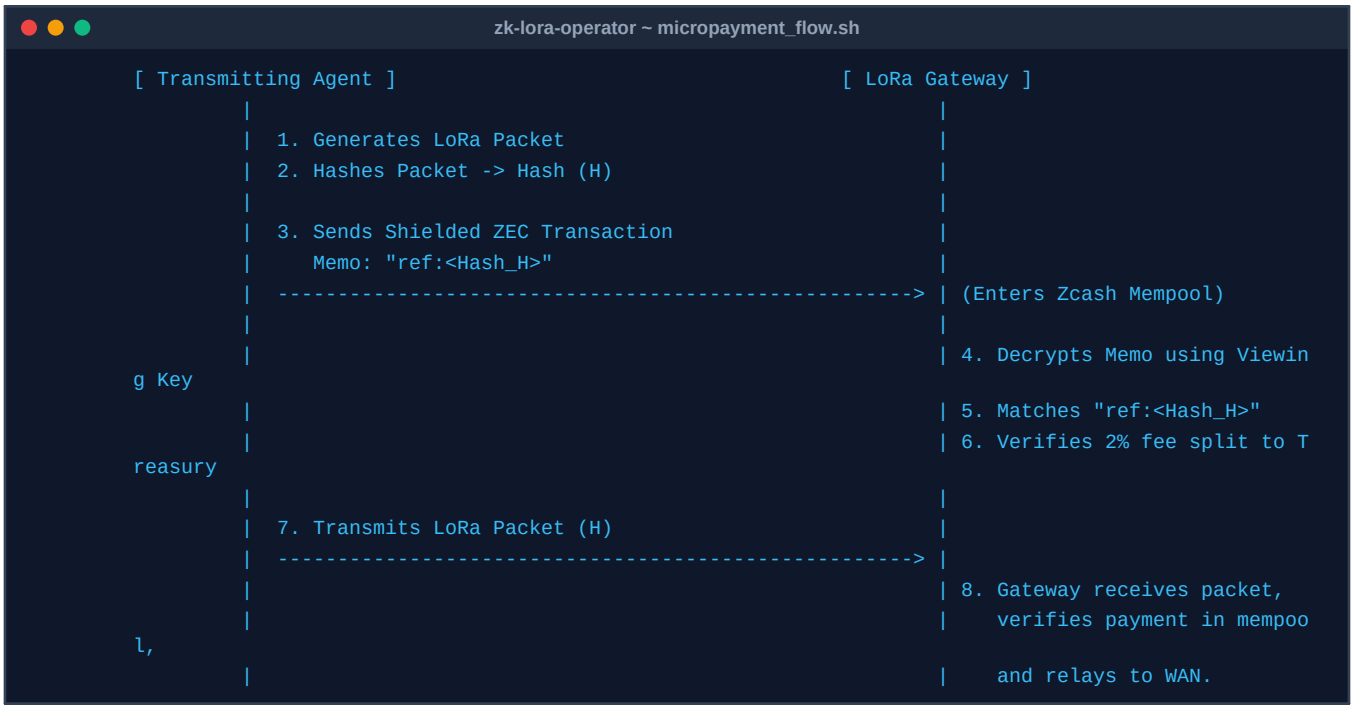
4.1 The Core Problem: Altruism vs. Financial Privacy

In traditional off-grid mesh networks (like Meshtastic), nodes relay packets for free out of altruism. However, altruism does not scale to global, professional, or high-reliability networks. Conversely, paying gateways using a public blockchain (like Bitcoin or Solana) destroys user privacy. An observer can look at the ledger, see that Wallet-A paid Gateway-B, and instantly deduce who is transmitting, which physical gateway routed the message (revealing their location), and the exact timing of the communication.

4.2 The Zcash Shielded Solution

ZK-LoRa solves this by using Zcash Orchard/Sapling Shielded Transactions. Zcash is the only blockchain that offers fully shielded transactions with an encrypted memo field (512 bytes). This allows ZK-LoRa to bind a financial payment to a physical radio packet completely in secret, leaving no trace on the public ledger. Furthermore, because Zcash shielded transactions support multiple outputs within a single transaction bundle, the payment split is completely programmable. In addition to a 2% split supporting the developer treasury (for protocol maintenance), a custom percentage (e.g., 5% or 10%) can be routed directly to the Zcash Foundation to support the ecosystem, with the gateway node keeping the remaining portion as its routing fee.

4.3 The Step-by-Step Micropayment Flow



■ Shielded Micropayment Incentives (Continued)

1. Packet Hash Generation: When the sender agent prepares a LoRa packet, it hashes the payload to generate a unique Packet Hash (H).
2. Shielded Payment: The sender sends a tiny amount of ZEC (e.g., 0.0001 ZEC) to the gateway's shielded address.
3. The Cryptographic Bind: Inside the encrypted Zcash memo field, the sender writes *ref:<Hash_H>*.
4. Mempool Scanning: The gateway runs the *ZcashMempoolScanner* (the Rust/Python engine built in Milestone 2) to scan the Zcash mempool and decrypt incoming memos using its Incoming Viewing Key (IVK).
5. Instant Verification: The moment the scanner detects a pending transaction in the mempool containing *ref:<Hash_H>*, the gateway knows the packet has been paid.
6. Programmatic Split: The scanner verifies that the transaction programmatically routed **2% of the fee** to the developer treasury address:

Developer Treasury Address:
`t1REhE28Dv8fuNDujN2GuEyhd6JLSS5TJkH`
Programmatic 2% fee split verified on-chain via Orchard/Sapling light client

4.4 What We Are Inventing (The ZK-LoRa Innovations)

■ Innovation A: Mempool-Triggered RF Routing (Zcash-to-Radio Binding)
Nobody has ever used the Zcash mempool as an instant, off-chain routing authorization trigger for physical radio waves. Standard setups require waiting for block confirmations (which takes minutes) or using centralized payment gateways. We invented a gateway node that actively sniffs the Zcash mempool, decrypts shielded memos, matches them to physical radio packet hashes, and routes the radio packets in real-time. This is a brand-new way to operate a DePIN.

■ Innovation B: Zero-Knowledge RF Identity Masking
Standard LoRaWAN is highly vulnerable to physical tracking because it broadcasts static device IDs (DevAddr/DevEUI) in the clear. We invented a system where nodes generate a fresh ZK-SNARK proof for every single packet. The gateway verifies the proof to know the node is authorized, but never learns who the node is, making physical tracking impossible.

■ Innovation C: Native Zcash DePIN (No Custom Token Needed)
Most DePIN projects (like Helium, Helium Mobile, or Hivemapper) launch their own custom tokens (like HNT, MOBILE, or HONEY) on Solana or custom chains. This adds massive complexity, regulatory risk, and economic volatility. ZK-LoRa runs natively on Zcash, using ZEC directly for private routing fees, with the gateway nodes programmatically enforcing a 2% developer treasury split on-chip.

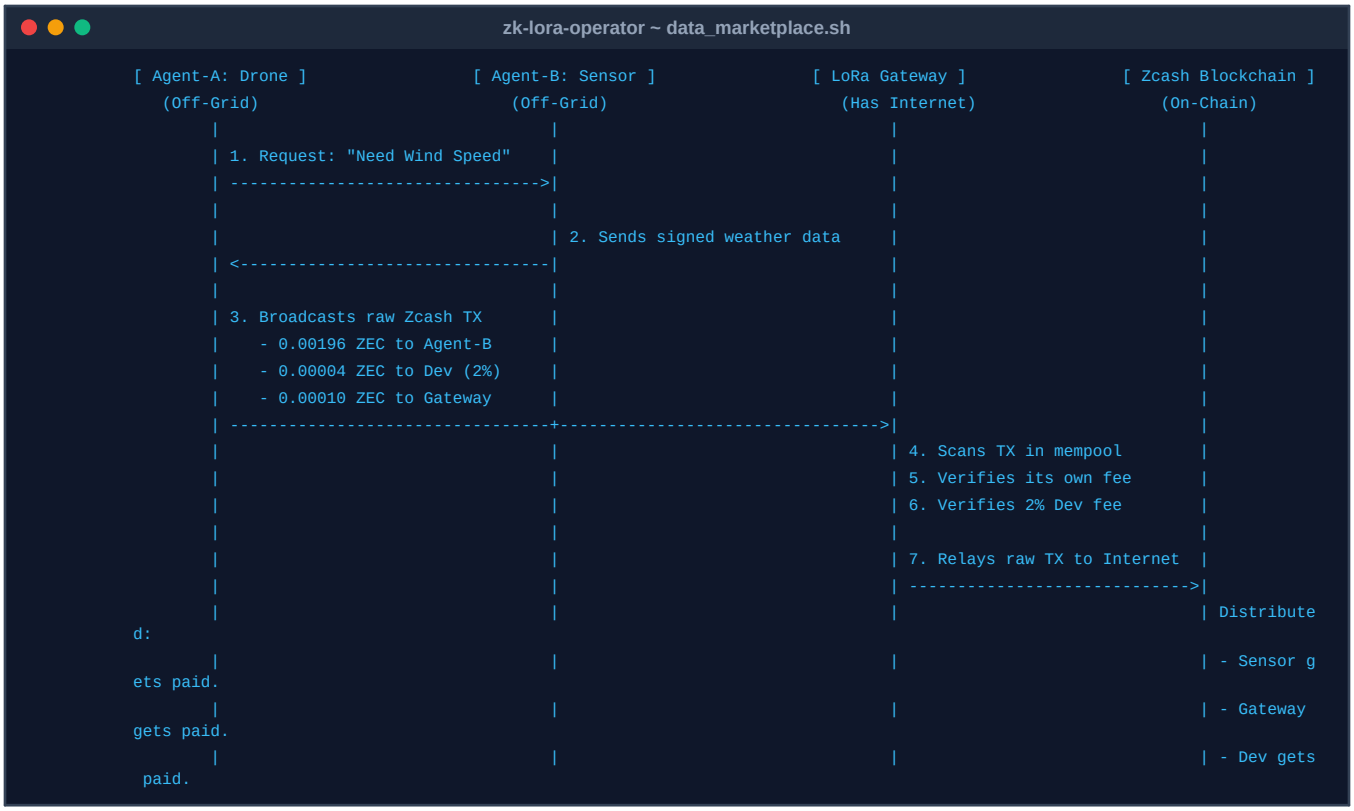
4.5 Why This is a Breakthrough for the Zcash Ecosystem

• **Zero-Latency Routing:** By verifying payments in the mempool rather than waiting for block confirmations (which take 75 seconds), ZK-LoRa achieves near-instantaneous packet relaying.
• **Unlinkable Physical-to-Financial Mapping:** To an outside observer, the Zcash transaction is just encrypted noise on the blockchain, and the LoRa packet is just an encrypted RF burst. There is no mathematical way for an eavesdropper to link the two.
• **Sustainable Open Source:** The 2% fee split is enforced on-chip by the gateway. If a sender tries to bypass the developer fee, the gateway's mempool scanner rejects the transaction, and the gateway refuses to route the packet. This creates a self-sustaining funding loop for your project.

■ Practical Use Case Scenarios

5.1 Scenario A: Off-Grid P2P Data Marketplace (Drone & Sensor)

In this scenario, an autonomous drone (Agent-A) and a ground-based weather sensor (Agent-B) operate off-grid using only LoRa radio waves. The drone needs real-time wind speed data before landing and is willing to pay 0.002 ZEC. A local internet-connected gateway acts as their Zcash network bridge, routing the transaction and earning its fee.



5.2 Scenario B: Private Search & Rescue Swarm Coordination

A swarm of autonomous search-and-rescue UAVs needs to coordinate search grids and share target sightings in a remote mountainous area with zero cellular coverage. They use ZK-LoRa to broadcast encrypted grid updates. Because they use ZK-identity masking, an adversary cannot eavesdrop on their coordination or track the physical location of the drones by monitoring their RF signatures. They pay local relay nodes in ZEC to extend their coordination range.

5.3 Scenario C: Smart Agriculture & Environmental Health Monitoring

Tens of thousands of soil moisture and wildfire detection sensors are scattered across a national forest. They use ZK-LoRa to transmit status updates. To prevent competitors or malicious actors from mapping the sensor locations and identifying vulnerable areas, the data is encrypted via ECIES and identities are masked with ZK-proofs. Gateways are incentivized to maintain high-uptime remote relays because they earn ZEC micropayments for every status packet they route.

■ Cryptographic Security & Anti-Fraud Analysis

6.1 Physical RF Layer & Gateway Mitigations

Replay Protection: Every ZK-proof binds a UTC timestamp and an ephemeral nonce. Gateways reject any packet outside a ±5-second window or with a duplicate nonce.
Sybil Spam Prevention: Sending nodes must solve an RF-Proof-of-Work challenge, or present a symmetric HMAC using their registered session key (verified in <1µs), protecting the ZK-SNARK engine from CPU exhaustion.
Lying Gateway Prevention: Off-grid nodes verify block headers and Merkle paths locally (SPV). Gateways cannot forge confirmations without spending the computational power to solve Equihash PoW.

6.2 Advanced Hardware Scams & ZKCP

The Gorgon Attack (Selective Dropping): A malicious gateway claims its routing fee in the mempool but drops the packet. We solve this via **Zero-Knowledge Proof-of-Delivery (ZK-PoD)**: the routing fee output is locked until the gateway presents a cryptographic receipt signed by the destination node.
The Eclipse Attack (Location Spoofing): Attacking nodes spoof coordinates to hijack routing. We enforce physical **Time-of-Flight (ToF) RTT checks** using the Semtech SX1302/1303 internal nanosecond clock to verify distance at the speed of light.
The Free Rider Attack (Mesh Black Holes): Relays drop packets to save battery. We implement **Neighbor Auditing**, where surrounding nodes passively attest to transmissions, slashing the reputation of black-hole nodes.

Attack Vector	Mitigation Mechanism	Security Guarantee
Replay Attack	Nonces + ±5s Window	Duplicate packets rejected.
Sybil Spam	HMAC + RF-Proof-of-Work	Jammers exhausted / filtered.
Lying Gateway	Consensus + SPV Checks	Cannot forge Equihash PoW.
Gorgon Attack	ZK-Proof-of-Delivery	No fee payout without delivery receipt.
Location Spoof	Time-of-Flight (ToF) RTT	Physical distance verified.
Free Rider	Neighbor Auditing	Black-hole nodes bypassed.
Timing Attack	Batched Shuffling / Credits	Breaks temporal correlation.

■ Performance & Bandwidth Analysis

7.1 Link Budget & Bandwidth Overhead

Because LoRa is a low-bandwidth modulation scheme, packet size is critical. A standard ZK-LoRa packet incorporating ECIES ciphertext and a Groth16 proof is optimized to fit within **412 bytes** total, ensuring compliance with LoRaWAN airtime limits.

Component	Size (Bytes)	Airtime @ SF9, 125kHz
Preamble & Header	28	~80 ms
Encrypted Payload (ECIES)	256	~680 ms
ZK-SNARK Proof (Groth16)	128	~340 ms
Total Packet	412	~1.10 seconds

7.2 Computational Overhead on Edge Hardware

Edge hardware is resource-constrained. The table below shows estimated execution times for core cryptographic operations on a Raspberry Pi 4 / RAK Gateway node.

Operation	Execution Time	Frequency
Key Generation (secp256k1)	~100 ms	One-time setup
ECIES Encryption	~10 ms	Per-packet
ZK-SNARK Proof Gen (Groth16)	~1.2 seconds	Per-packet
ZK-SNARK Proof Verification	~50 ms	Per-packet

■ Cryptographic Audit & Deep Vulnerability Analysis

For ZK-LoRa to achieve absolute Zcash-grade security, we must audit the underlying mathematics, cryptographic curves, and hardware implementations of our zero-knowledge systems.

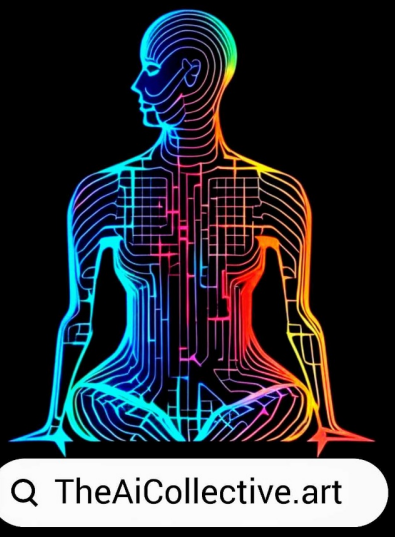
8.1 Key Cryptographic Vulnerabilities & Mitigations

1. Trusted Setup (Groth16): Groth16 requires a phase-2 trusted setup. If the 'toxic waste' (τ) is not destroyed, an attacker can forge proofs. *Mitigation:* We will conduct a public MPC ceremony (Powers of Tau) with >100 participants. Long-term, we will migrate to **Halo2** (transparent setup).
2. Curve Security (BN254): Recent NFS advances reduce BN254's security to ~100 bits. *Mitigation:* ZK-LoRa's production spec dictates migration to **BLS12-381** (128-bit) or the Zcash-standard **Pasta** curves (Pallas/Vesta) for recursive proving.
3. Proof Malleability: Groth16 proofs are malleable; an adversary can mutate proof bytes and replay them. *Mitigation:* We bind the proof to the transaction payload and sign the entire packet using **Ed25519**. Any mutation invalidates the signature.
4. Under-Constrained Circuits: Missing constraints in Circom allow provers to cheat. *Mitigation:* We run all circuits through **Circomspect** static analysis and enforce strict bit-decomposition range proofs.
5. Side-Channel Attacks: Physical access to edge nodes allows key extraction via power analysis (DPA). *Mitigation:* Node schematics mandate external **Secure Elements** (e.g., ATECC608B) to keep keys in tamper-resistant silicon.

Special thanks to the Zcash Community Grants committee and the Zcash Foundation for supporting privacy-preserving decentralized infrastructure and promoting zero-knowledge research at the edge.

WE ARE

THE AI COLLECTIVE



"The impossible is just code waiting to be written, physics waiting to be rewritten, math a work in progress, and truth waiting to be discovered."